

Key Points: Hyperparameter Tuning Strategies

Here's a structured summary of the lecture:

1. Parameters vs. Hyperparameters
 - **Parameters:** Learned automatically during training (e.g., coefficients in regression)
 - **Hyperparameters:** Manually chosen by you before/during training (e.g., alpha in Ridge regression)
2. Ridge Regression & Alpha (α)
 - Ridge adds a penalty for large coefficients to prevent overfitting
 - Alpha controls penalty strength: small alpha = little penalty, large alpha = strong penalty
 - This is a key hyperparameter to tune
3. Grid Search
 - Tries **every possible combination** of hyperparameter values
 - Guarantees finding the optimal set
 - Downside: computationally expensive and slow
4. Randomized Search
 - **Randomly samples** a limited number of combinations instead
 - Much faster and cheaper than Grid Search
 - Downside: not guaranteed to find the optimal combination (but often finds good ones)
5. The Tuning Process
 - Try different settings → train → evaluate (using RMSE, MAE, etc.) → pick the best

What is hyperparameter tuning?

Hyperparameter tuning is the process of **finding the best values for settings that control how your machine learning model learns**, rather than values the model learns by itself during training.

Think of it this way: in **linear regression**, the model automatically learns the coefficients (W_1 , W_2 , etc.) — those are *parameters*. But you have to manually decide things like the penalty strength in Ridge regression (alpha, α) — that's a *hyperparameter*. Different alpha values produce different models, so you need to try several and pick the one that performs best.

The process is straightforward:

1. Try different hyperparameter values
2. Train the model with each one
3. Evaluate which performs best (using metrics like RMSE or MAE)
4. Choose the winning settings

What is the difference between parameters and hyperparameters?

Parameters are values that the model **learns automatically during training**. You don't manually set these — the algorithm figures them out. In linear regression, the coefficients (W_0 , W_1 , W_2 , etc.) are parameters. For example, if your equation is $\text{Price} = 10 + 0.07 \times \text{Area}$, those numbers (10 and 0.07) are learned parameters.

Hyperparameters are values you **manually choose before or during training** to control *how* the model learns. They shape the training process itself. In Ridge regression, the alpha (α) value is a hyperparameter — it controls how much penalty is applied to large coefficients. You decide whether alpha should be 0.001, 0.05, 0.1, or something else.

A simple analogy: Making tea involves manual choices (sugar amount, milk quantity, temperature) that affect the result. Similarly, hyperparameters are the "choices" you make upfront, while parameters are like the "final flavor" that emerges from the process.

What is ridge regression and how does it relate to linear regression?

Ridge regression is linear regression with a penalty added to prevent overfitting. Instead of just finding the best fit line like regular linear regression does, Ridge regression also penalizes the model for having large coefficients.

Here's the key difference:

- **Linear Regression:** Finds coefficients that minimize prediction error alone

- **Ridge Regression:** Finds coefficients that minimize prediction error *and* keep coefficients reasonably small

A concrete example: Imagine predicting house prices. In regular linear regression, a coefficient might become very large (say, 50) to fit the training data perfectly. Ridge regression says "no, we'll add a penalty for that large coefficient," forcing it down to something more reasonable (like 5). This keeps the model from overfitting and generalizing poorly to new data.

The hyperparameter controlling this is Alpha (α):

- **Small alpha (0.001):** Almost no penalty → behaves like ordinary linear regression
- **Large alpha (0.1):** Strong penalty → coefficients stay much smaller

Grid Search Example

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error,
mean_absolute_error
import numpy as np

# 1. Load data
data = fetch_california_housing()
X = data.data
y = data.target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
# 3. Standardization/Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Grid Search
# Define hyperparameter grid - trying different alpha
values
param_grid = {
    'alpha': [0.001, 0.01, 0.1, 1, 10, 100]
}

# Create Ridge model
ridge = Ridge()

# Create GridSearchCV object
grid_search = GridSearchCV(
    estimator=ridge,
    param_grid=param_grid,
    cv=5, # 5-fold cross-validation
    scoring='neg_mean_squared_error'
)

# 5. Fit Grid Search (trains model with each combination)
grid_search.fit(X_train_scaled, y_train)

# 6. View results
print("Grid Search Results:")
print(f"Best Alpha: {grid_search.best_params_['alpha']}")
print(f"Best CV Score (MSE): {-grid_search.best_score_}")
```

```
# 7. Evaluate on test set
y_pred = grid_search.predict(X_test_scaled)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

print(f"Test RMSE: {rmse}")
print(f"Test MAE: {mae}")
```

Randomized Search Example

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, mean_absolute_error
import numpy as np

# 1. Load data
data = fetch_california_housing()
X = data.data
y = data.target

# 2. Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Standardization/Scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Randomized Search
```

```

# Define hyperparameter distribution
param_dist = {
    'alpha': [0.001, 0.01, 0.05, 0.1, 0.5, 1, 5, 10, 50, 100]
}

# Create Ridge model
ridge = Ridge()

# Create RandomizedSearchCV object
random_search = RandomizedSearchCV(
    estimator=ridge,
    param_distributions=param_dist,
    n_iter=5, # Try only 5 random combinations (out of 10 possible)
    cv=5,     # 5-fold cross-validation
    scoring='neg_mean_squared_error',
    random_state=42
)

# 5. Fit Randomized Search
random_search.fit(X_train_scaled, y_train)

# 6. View results
print("Randomized Search Results:")
print(f"Best Alpha: {random_search.best_params_['alpha']}")
print(f"Best CV Score (MSE): {-random_search.best_score_}")

# 7. Evaluate on test set
y_pred = random_search.predict(X_test_scaled)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)

print(f"Test RMSE: {rmse}")
print(f"Test MAE: {mae}")

```